Hash Tables in Java

Mostafa Faik

October 20, 2024

Introduction

In this assignment, we explored different methods to organize a set of entries (`Swedish zip codes`) to efficiently search and retrieve entries based on their `zip code`. We began with a simple, inefficient solution, progressively improving it by examining both `space` and `time efficiency`. Starting with `linear search`, we moved to `binary search`, and finally to `hashing techniques` with and without `collision handling`. The goal was to balance the trade-off between speed and memory usage while maintaining fast access to the data.

We used a dataset of *9,675* `Swedish zip codes`, containing the `zip code`, `area name`, and `population`, and tested the performance of different search methods by `benchmarking` the time it takes to find specific `zip codes`.

Loading Zip Code Data and Basic Search Methods

We started by loading the `zip code` data from a CSV file into an array of objects. Each entry contains three fields:

- Zip Code (string or integer)
- Area Name
- Population

The file is read and stored in an array. We implemented two search methods:

- `Linear Search`: This method iterates through the array, comparing each entry with the target `zip code`.
- `Binary Search`: Since the `zip codes` are ordered, `binary search` divides the array into halves, reducing the search space each time.

CSV File Handling:

The `CSV file` is parsed using `Java's BufferedReader`. Each line is split by commas to extract the `zip code`, `area name`, and `population`. These values are then stored in an array of Area objects, which contains each entry.

The following are the results of the benchmarks for both linear and binary search:

Method	**Zip Code**	**Time (ns)**	**Result**
Linear Search	"111 15"	10,100	STOCKHOLM
Linear Search	"984 99"	62,700	PAJALA
Binary Search	"111 15"	7,100	STOCKHOLM
Binary Search	"984 99"	5,500	PAJALA

Table 1: String-Based Search

Method	**Zip Code**	**Time (ns)**	**Result**
Linear Search	11115	7,000	STOCKHOLM
Linear Search	98499	34,800	PAJALA
Binary Search	11115	5,500	STOCKHOLM
Binary Search	98499	2,600	PAJALA

Table 2: Integer-Based Search

We can observe from the two tables that:

- `Linear Search`: As expected, `linear search` was slow for large datasets. The time taken for "*984 99*" (near the end of the dataset) was significantly higher than for "*111 15*", because of the time complexity $O(n)$.
- `Binary Search`: The `binary search` method performed faster due to its $O(logn)$ complexity, especially with integer-based comparisons, where performance improved further.

`String vs Integer Comparison`:

- Using `integers` for the `zip codes` (instead of `strings`) reduced the `search time` further, particularly for `binary search`. Converting `zip codes` to `integers` improves execution speed because `integer` comparisons are faster than `string` comparisons.
- `Numeric Comparisons`: Converting the `zip codes` to `integers` led to a minor but noticeable improvement in `search speed`. Since `integers` are more efficient to compare than `strings`, this optimization is worth considering for larger datasets.

Using Zip Code as Index in an Array

In this task we tried a different approach. Since the `zip codes` are `numeric`, we used the `zip code` itself as an index in a large array. We created an array with *100,000* elements (since the largest `zip code` is *99999*) and directly mapped each `zip code` to its corresponding index.

After reading the `CSV file, zip codes` were stripped of spaces and converted into `integers`. These `integers` were used as direct indices in an array, allowing constant-time lookup of `zip codes`.

Zip Code	**Time (ns)**	**Result**
11115	9,300	STOCKHOLM
98499	2,500	PAJALA

Table 3: Benchmark Results

This method allows for `constant-time lookup` (i.e., $O(1)$), where the `zip code` serves as a direct index to access the corresponding entry.

We can also observe that direct array lookup for `zip codes` was extremely fast, with `lookup times` much faster than `binary search`, as expected. However, this approach required a large amount of memory, since most of the array was empty.

Space Efficiency: While the `lookup speed` was excellent, the memory usage was inefficient, as most of the array was unused. This approach may not be feasible for large datasets with `sparse keys`.

Optimizing Space with Hashing

To address the space inefficiency, we introduced a `hash function`. The idea was to map each `zip code` to a smaller array by using the `zip code` *modulo* a value m. By choosing an appropriate m, we can significantly reduce the size of the `array` while minimizing `collisions`.

We measured the number of `collisions` (multiple `zip codes` mapping to the same index) for different *modulo* values. The experiment tracked the number of indices with two, three, or more `collisions`.

Modulo Value	2 Keys	3 Keys	4 Keys	5 Keys	6+ Keys
10000	2050	1130	545	334	203
20000	4180	1471	509	194	50
12345	4997	1804	311	33	1
17389	6352	1414	161	3	0
13513	5410	1678	287	12	0
13600	3406	1578	613	229	56
14000	3055	1316	615	320	134

Table 4: Collision distribution for different modulo values

`Collisions`: We observed that the number of `collisions` varied depending on the *modulo* value. Using values like *13,513* and *17,389* resulted in fewer `collisions` compared to more predictable values like *10,000* or *20,000*.

`Hash Function Trade-offs`: Finding the right `hash function` is a trade-off between minimizing `collisions` and using space efficiently. Larger values of m tend to reduce `collisions` but may result in wasted space. We found that *13,513* was a good balance between space efficiency and `collision` reduction, which offered fewer `collisions` without consuming excessive space.

Handling Collisions with Buckets

To handle `collisions` in the `hash table`, we implemented `buckets`. Each index in the array corresponds to a `small list` of entries that share the same `hash value`. When a `collision` occurs, the new entry is added to the list (`bucket`) at that index. When performing `lookups`, we iterate through the `bucket` to find the correct entry.

Zip Code	Result	Population
11115	STOCKHOLM	3
98499	PAJALA	200
99999	Not Found	-

Table 5: Benchmark Results

`Bucket-based Lookup` was fast and efficient, even with `collisions`. The lookup time remained close to `constant`, with only a small performance hit when `buckets` contained multiple entries. Even with `collisions`, the time overhead for searching through a small `bucket` is minimal.

Efficient Collision Handling: By using small `buckets` to handle `collisions`, we maintained fast lookup times while significantly reducing memory usage compared to the direct index approach. This approach provides a good balance between time and space efficiency.

Improving Buckets with Linear Probing

To further optimize the `handling of collisions`, we implemented `linear probing`. Instead of using `buckets`, when a `collision` occurred, we checked the next available slot in the `array`. If that slot was also occupied, we continued searching until an empty slot was found. Similarly, during `lookups`, we searched from the `hash value` index forward until we found the correct entry or encountered an empty slot.

Array Size	**Zip Code**	**Probes**
10000	11115 (STOCKHOLM)	1
10000	98499 (PAJALA)	1709
20000	11115 (STOCKHOLM)	1
20000	98499 (PAJALA)	18
30000	11115 (STOCKHOLM)	1
30000	98499 (PAJALA)	4
40000	11115 (STOCKHOLM)	1
40000	98499 (PAJALA)	5
50000	11115 (STOCKHOLM)	1
50000	98499 (PAJALA)	1

Table 6: Linear Probing Results for Various Array Sizes

- `Linear Probing`: As expected, `linear probing` was efficient for larger `arrays`. For smaller `arrays` (e.g., *10,000*), more `probes` were needed to find the correct entry in the case of `collisions`, as shown by the *1,709* `probes` for the "*98499*" `zip code`.
- `Probes Count`: The number of `probes` required decreased as the `array` size increased. For example, with an `array` size of *50,000*, only *1* `probe` was needed for both `zip codes`, showing that larger `arrays handle collisions` more efficiently.
- `Space vs Time Trade-off`: `Linear probing` works well when the `array` is large enough, but if the `array` becomes too full, the number of `probes` can increase significantly, leading to slower `lookup times`.

Conclusion

This assignment explored various techniques for organizing and searching a table of `Swedish zip codes`. Starting from simple `linear search`, we progressively optimized the lookup process through `binary search`, using `zip codes` as direct indices, and finally applying `hashing` techniques with `collision handling`.

- `Linear and Binary Search`: `Binary search` significantly outperformed `linear search` due to its $O(logn)$ `time complexity`, especially when using `integers` for comparisons.
- `Direct Indexing`: Using the `zip code` as an index provided `constant-time lookup` ($O(1)$) but was highly inefficient in terms of memory usage, as most of the array was empty.
- `Hashing`: `Hash-based lookup` with well-chosen `modulo` values provided a balance between time and space efficiency. `Modulo` values like *13,513* offered fewer `collisions` while keeping the `array` size manageable.
- `Collision Handling`: `Bucket-based collision handling` worked well to resolve `collisions` without significantly increasing `lookup times`. `Linear probing` further optimized this process, but its efficiency depended heavily on `array size`.

In the end, the choice of method depends on the specific trade-offs between time and space efficiency that the application requires. For small datasets with well-distributed keys, using `hash-based` methods with `buckets` provides excellent performance. For larger datasets with more sparse keys, direct indexing may be preferred if memory usage is not a concern.